

Reasoning Primitives in Hybrid and Non-Hybrid LLMs

Anonymous ACL submission

Abstract

Reasoning in large language models is often treated as a monolithic capability, but its observed gains may arise from more basic operations. We study reasoning through two such primitives which are recall and state-tracking and ask whether hybrid architectures combining attention-based retrieval with recurrent state updates are better suited than attention-only models on tasks that jointly require both. Using matched OLMo3 transformer and hybrid models in instruction-tuned and reasoning-augmented variants, we evaluate five controlled task families stressing state-based recall. Reasoning augmentation clearly and consistently improves performance across all tasks, often by a large margin, aligning with the *State over Tokens* interpretation: because state management largely happens in token space, externalised reasoning traces are highly effective. The hybrid architecture’s additional inductive bias for stateful computation does not reliably translate into accuracy gains beyond what reasoning tokens already provide, however where a hybrid advantage does appear, it is conditional and task-dependent. On tasks with strictly sequential chained state updates the hybrid Think model is more robust at high difficulty; on tasks requiring flat multi-hop retrieval the transformer Think model is more robust. Given the small scope of this case study, we present these findings as suggestive rather than conclusive.

1 Introduction

Early work on Chain-of-Thought reasoning established a now-familiar empirical fact: when large language models (LLMs) are induced to generate intermediate steps, they often perform substantially better on complex tasks (Wei et al., 2022). Subsequent work expanded this picture across prompting strategies (Yao et al., 2023), architectural settings (Geiping et al., 2025), and training regimes (Wang et al., 2025; Yang et al.,

2025; Bakouch et al., 2025; Zeng et al., 2026). The harder question is not whether reasoning tokens help, but why they help.

A broad literature treats reasoning in LLMs as structured inference unfolding through intermediate textual computation (Liu et al., 2025), but the mechanisms remain contested (Kambhampati et al., 2025). Recent work (Levy et al., 2025) proposes the **State over Tokens** (SoT) paradigm: autoregressive models are applied iteratively, but internal activations do not persist across generation steps, what persists is only the token prefix. Intermediate tokens therefore serve as the only durable carrier of state across successive forward passes. The gain from reasoning does not arise merely because the model explains itself; it arises because additional tokens let it externalize partial results, recondition on them, and transform a sequence of bounded computations into a cumulative one. This also explains a familiar puzzle: reasoning traces often look coherent without being faithful descriptions of the underlying computation. Under the SoT view, intermediate text needs only to encode information useful for subsequent computation, not to transparently represent the model’s internal process. A natural extension of this framing is to ask whether models could be trained to use highly compressed, non-linguistic token formats optimised for state encoding rather than human readability and whether such formats would match or exceed the performance of natural language traces. There is already evidence that this is feasible (Song et al., 2026), though we leave such tests to future work and return to their implications in Section 5.

Once reasoning is framed in these terms, the question of architecture becomes central. Standard softmax-based transformers excel at content-based retrieval but face well-known scaling constraints: full self-attention requires $O(n^2)$ computation and memory, and even with KV caching, sequence length remains a bottleneck as memory grows with stored keys and values (Fichtl et al., 2025; Li et al., 2024). Recurrent and state-space architectures offer more favorable asymptotic profiles but sacrifice precise content-addressable retrieval (Yang et al., 2024b,a; Dao and Gu, 2024). This tension motivates *hybrid architectures*, which

combine both mechanisms (Merrill et al., 2026). In their account, attention supports fine-grained recall over static context, whereas recurrent components support persistent, efficiently updated latent state. These are complementary inductive biases rather than competing trade-offs.

These observations point toward a concrete hypothesis: if reasoning tokens improve performance by externalizing state, then the relevant underlying capabilities are not “reasoning” in the abstract but more basic operations enabling stateful computation. In this work, we focus on two such primitives **Recall** and **State-tracking** and ask under what conditions architectural differences become visible once reasoning is decomposed into its constituent computational demands.

2 Recall and State-Tracking as Reasoning Primitives

To make the notion of reasoning operational, we decompose it into two complementary computational primitives following (Merrill et al., 2026): **State-tracking**, maintaining and updating structured variables across sequential transformations, where recurrent models are well matched because they update a compact hidden state incrementally and **Recall**, retrieving specific information from a large context with high precision, where softmax attention functions as an associative memory mechanism (see Appendix E for formal definitions). When combined into **State-based recall**, the retrieval address is itself determined by a sequence of state transformations, making the task irreducible to either primitive alone: pure attention lacks stable iterative state evolution; pure recurrence compresses away the fine-grained addressability needed for precise lookup. See Appendix B for canonical examples.

3 Methods and Tasks

We evaluate matched transformer and hybrid models, with and without reasoning augmentation, on five procedurally generated tasks designed to stress state-based recall. Each task varies along two difficulty axes: m (retrieval complexity) and n (state maintenance complexity).

Models. We compare OLMo3-7B (Olmo et al., 2025) and OLMo-Hybrid-7B (Merrill et al., 2026), a matched pair sharing the same data mixture, optimization schedule, and training recipe, a near-ideal architectural ablation (full architecture details in Appendix G). We evaluate both instruction-tuned and reasoning-augmented variants: OLMo-3-7B-`{Instruct,Think}` and OLMo-Hybrid-`{Instruct,Think}`-SFT-7B. This set lets us examine two questions simultaneously: (1) Does explicit reasoning-oriented training

Task	Description	m	n
OLMo Original	Bit value after pointer swaps in a synthetic array	Array length	# swaps
Astro Recall	Planet name after variable swaps on orbital periods	# table rows	# swaps
Collisions	Final velocity after elastic collision chain	# particles	# collisions
Dyck Language	Correct closing bracket in a masked sequence	Stack depth	Sequence length
DAG Arithmetic	Variable value after layered arithmetic with long-range dependencies	# variables	# layers

Table 1: Task descriptions and difficulty parameters. m controls retrieval complexity; n controls state maintenance complexity. Tasks are ordered from most minimal (OLMo Original) to most complex. Full details in Appendix C.

improve performance on tasks requiring multi-step state manipulation? (2) Does the effect depend on architectural class? Full inference configuration details are provided in Appendix F.3.

Evaluation. All tasks are free-form generation scored by exact matching. Difficulty is parameterised by m (retrieval complexity) and n (state maintenance complexity), with 1000 instances per (m, n) bin (500 for Dyck). All tasks require JSON output `{"answer": "A|B|C|D"}`. We report *accuracy* and *parse rate* (fraction of parseable responses) separately.

Tasks. We evaluate five procedurally generated task families, each a different surface realisation of state-based recall varying in their mix of recall and state-tracking demands (Table 1; full details in Appendix C). **OLMo Original** (Merrill et al., 2026) is the minimal replication baseline; the remaining four are our own contributions: **Astro Recall** (pointer-tracking with table lookup), **Collisions** (chained sequential state updates), **Dyck** (pure state-tracking, no retrieval), and **DAG Arithmetic** (flat multi-hop retrieval over a growing registry (Motwani et al., 2026)).

4 Results

4.1 OLMo Original: Replication Baseline

Under instruction-only prompting, the transformer Instruct model consistently outperforms the hybrid Instruct model across all difficulty levels (0.43–0.53

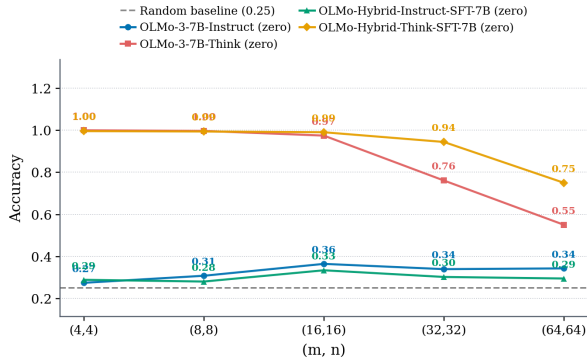


Figure 1: Collision Simulator accuracy.

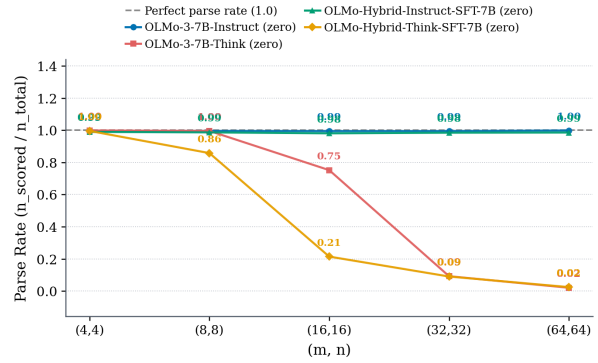


Figure 2: Collision Simulator parse rate.

vs. 0.32–0.43), despite neither being far above the 0.25 random baseline; the gap is consistent from (4, 4) to (64, 64) and does not widen or close with difficulty. The Think variants present a different pattern: both achieve near-perfect accuracy at (4, 4) (transformer Think 0.91, hybrid Think 0.92) and degrade monotonically in parallel toward chance by (64, 64), with no consistent architectural advantage in either direction (Figure 9). The parse rate (Figure 10) shows that the transformer variants produce more parseable outputs than their hybrid counterparts at both training regimes. At the Think level, however, this formatting advantage does not translate into accuracy differences.

4.2 Collision Simulator

Under instruction-only prompting, both architectures hover between 0.27–0.40, with neither consistently exceeding chance. At low difficulty both Think variants perform near ceiling. Once difficulty increases beyond (16, 16) the two architectures diverge: OLMo-3-7B-Think falls to 0.49 at (32, 32) and increasingly exhausts its inference budget without producing a parseable final answer at (64, 64), while OLMo-Hybrid-Think-SFT-7B retains 0.94 and 0.75 at the same settings (Figure 1). The parse rate figure (Figure 2) reveals a pattern similar with what is observed on **OLMo Original**: despite its higher accuracy, the hybrid Think model’s parse rate falls below the transformer’s at high difficulty. Fewer of its outputs are parseable, but a higher proportion of those that are contain the correct answer while opposite is true for transformer Think model.

4.3 DAG Arithmetic

Under instruction-only prompting, both Instruct variants begin above chance at (2, 2) (transformer Instruct 0.57, hybrid Instruct 0.51) and degrade steadily toward the 0.25 random baseline by (32, 32), tracking each other closely throughout with no consistent architectural signal between them. The Think variants tell a sharply different story: both achieve perfect accuracy (1.00) from

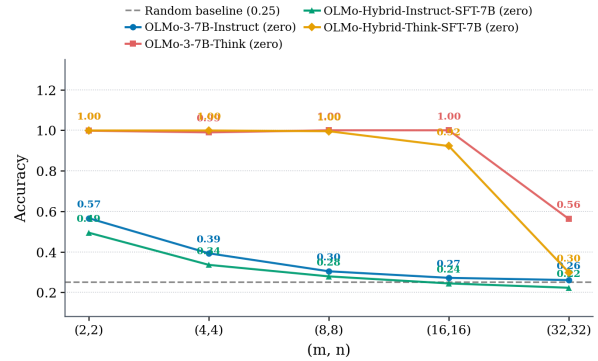


Figure 3: DAG Arithmetic accuracy.

(2, 2) through (8, 8). Beyond (16, 16) the architectures diverge in the opposite direction to collisions: OLMo-3-7B-Think retains 0.56 at (32, 32) while OLMo-Hybrid-Think-SFT-7B falls to 0.30 (Figure 3). The parse rate figure (Figure 4) adds an important contrast with the collision task: parse rates decline for both Think models as difficulty increases, but the hybrid Think model parses more reliably than the transformer Think model throughout, confirming that the accuracy gap reflects genuine computational failure rather than budget exhaustion.

4.4 Astro Recall

Under instruction-only prompting, both Instruct variants hover near chance throughout all difficulty levels, maintaining near-perfect parse rates, consistent with the pattern seen on other tasks where Instruct models produce well-formed but computationally uninformative outputs. The Think variants tell a different story: both achieve ≈ 0.95 –0.97 at (4, 4) but fail to produce parseable answers within budget by (32, 32). Even though the accuracy drops for both of them for transformer Think goes to in contrast to hybrid Think which drops till random baseline. The parse rate figure reveals that this failure is generational: both Think models’ parse rates collapse sharply with difficulty, with the hybrid Think model degrading faster, reaching near zero by (32, 32), while the transformer Think

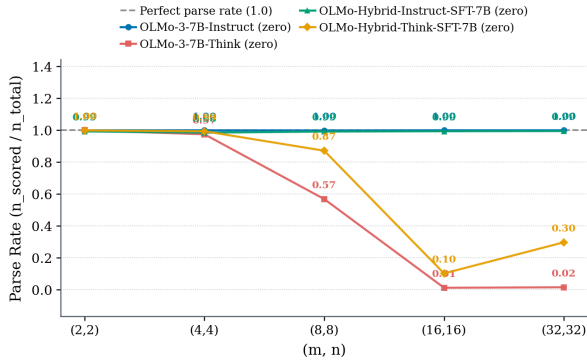


Figure 4: DAG Arithmetic parse rate.

model sustains a slightly higher parse rate before also collapsing. Whether accuracy would improve with a larger inference budget is an open question that computational constraints precluded us from testing. See Appendix A, Figures 5, 6.

4.5 Dyck Language

Under instruction-only prompting, both Instruct variants hover between 0.33–0.43 across all (m, n) settings, barely above the 0.33 random baseline, with no consistent difference between them. The Think variants diverge sharply: OLMo-Hybrid-Think-SFT-7B achieves the highest accuracy of any model in the study (peak 0.79 at $(2, 128)$, never falling below 0.38 at any tested setting), while OLMo-3-7B-Think is substantially more volatile (peak 0.67, dip to 0.22 at $(4, 2048)$). The parse rate figure adds a pattern consistent with what is observed on the collision task: the hybrid Think model’s parse rate degrades at high difficulty while its accuracy remains comparatively high, meaning fewer outputs are parseable but a higher proportion of those that are contain the correct answer. See Appendix A, Figures 7, 8.

5 Discussion and Conclusion

Across all five tasks, four consistent patterns emerge that together characterise the relationship between reasoning augmentation, architectural inductive bias, and task structure.

Reasoning augmentation is the dominant factor. Think variants substantially outperform Instruct variants on all computation-heavy tasks at low-to-moderate difficulty, consistent with the SoT account that externalizing intermediate state into tokens enables deeper cumulative computation. This benefit is largely architecture-agnostic in the moderate regime, both transformer and hybrid Think models benefit roughly equally from explicit reasoning traces, and the hybrid architecture’s additional inductive bias for stateful computation does not reliably translate into accuracy gains beyond what reasoning tokens already provide.

The hybrid advantage is conditional and task-dependent, not uniform. The OLMo Original results are particularly important here: on the task most directly imported from prior work and most theoretically motivated as a test of hybrid advantage, the two Think models are indistinguishable. This shows that conclusions from the OLMo3 Hybrid paper do not extrapolate straightforwardly to this setting. Where a hybrid advantage does appear—on collisions and Dyck—it is tied to specific task computational structures rather than being a general property of the architecture.

Chained sequential updates and flat multi-hop retrieval favour different architectures. The collision and DAG arithmetic results suggest that architectural differences depend on task structure rather than difficulty alone. On collisions, where updates are strictly sequential, the hybrid Think model remains substantially more robust in both accuracy and parse rate. On DAG arithmetic, the transformer Think model retains higher accuracy, indicating an advantage on flat, content-addressable retrieval over a growing variable registry. Taken together, the results suggest complementary architectural affordances rather than a universal ranking: recurrent components favour chained state maintenance, while transformer attention favours large-scale associative retrieval.

Where hybrids show an advantage, it may reflect reduced inference cost rather than improved expressivity. The Dyck result is suggestive of recurrent components supporting stack-based computation, but remains too narrow for strong conclusions. More broadly, hybrid architectures may derive their practical advantage not from greater expressivity, but from reducing the inference cost of maintaining long-range state internally rather than externalizing it entirely into reasoning tokens. In this view, sustained parse rates under high difficulty reflect more efficient state propagation rather than fundamentally stronger computation.

The main contribution of this work is less a definitive ranking than a more precise framing of the problem. Reasoning tokens may extend effective computation, but their benefit likely depends on how well the underlying architecture supports persistent state construction. Because some runs approached seven days of compute, systematic inference-budget scaling was infeasible here. Given the narrow empirical scope, the findings are suggestive rather than conclusive. Future work should test the SoT hypothesis more directly using compressed, non-linguistic reasoning traces (Song et al., 2026) and benchmarks grounded in recall and state-tracking primitives.

6 Limitations and Future Work

All experiments use a single matched OLMo3 pair at 7B; whether the observed patterns replicate across other hybrid architectures (Mamba, RWKV), other transformer families, or other parameter scales remains open. Relative advantages and the difficulty thresholds at which reasoning augmentation fails are likely scale-dependent, and testing the same task families at 1B, 13B, and 70B would establish whether the conditional hybrid advantages are specific to OLMo3. The five task families probe recall, state-tracking, and their composition in synthetic settings and do not license claims about general reasoning ability; extending evaluation to more naturalistic settings such as long-document question answering or multi-step mathematical derivations would test whether the primitives identified here remain predictive outside the controlled regime. All models are evaluated under zero-shot prompting, making it impossible to determine how much of the observed gap reflects genuine architectural differences versus distributional mismatch; fine-tuning experiments would allow a cleaner test. Two further directions are worth highlighting: the parse-rate/accuracy dissociation raises the question of whether evaluation methods that probe internal representations directly would change the conclusions; and, given that our results are broadly consistent with the SoT account, it would be valuable to test whether models that perform reasoning in latent space rather than token space can sustain accuracy at higher difficulty without the budget exhaustion we observe, a natural next step as such models become available.

References

- Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patino, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, and 1 others. 2025. Smollm3: smol, multilingual, long-context reasoner. *Hugging Face Blog*.
- Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL <https://arxiv.org/abs/2307.08691>, 5(10).
- Tri Dao and Albert Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*.
- Alexander M Fichtl, Jeremias Bohn, Josefin Kelber, Edoardo Mosca, and Georg Groh. 2025. The end of transformers? on challenging attention and the rise of sub-quadratic architectures. *arXiv preprint arXiv:2510.05364*.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*.
- Subbarao Kambhampati, Kaya Stechly, and Karthik Valmeekam. 2025. (how) do reasoning models reason? *Annals of the New York Academy of Sciences*, 1547(1):33–40.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.
- Mosh Levy, Zohar Elyoseph, Shauli Ravfogel, and Yoav Goldberg. 2025. State over tokens: Characterizing the role of reasoning tokens. *arXiv preprint arXiv:2512.12777*.
- Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen. 2024. A survey on large language model acceleration based on kv cache management. *arXiv preprint arXiv:2412.19442*.
- Hanmeng Liu, Zhizhang Fu, Mengru Ding, Ruoxi Ning, Chaoli Zhang, Xiaozhang Liu, and Yue Zhang. 2025. Logical reasoning in large language models: A survey. *arXiv preprint arXiv:2502.09100*.
- William Merrill, Yanhong Li, Tyler Romero, Anej Svete, Caia Costello, Pradeep Dasigi, Dirk Groeneveld, David Heineman, Bailey Kuehl, Nathan Lambert, and 1 others. 2026. Olmo hybrid: From theory to practice and back. *arXiv preprint arXiv:2604.03444*.
- Sumeet Ramesh Motwani, Daniel Nichols, Charles London, Peggy Li, Fabio Pizzati, Acer Blake, Hasan Hammoud, Tavish McDonald, Akshat Naik, Alesia Ivanova, and 1 others. 2026. Longcot: Benchmarking long-horizon chain-of-thought reasoning. *arXiv preprint arXiv:2604.14140*.
- Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, and 49 others. 2025. *Olmo 3. Preprint*, arXiv:2512.13961.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca

Antiga, and 1 others. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.

Jiwon Song, Dongwon Jo, Yulhwa Kim, and 1 others. 2026. Reasoning path compression: Compressing generation trajectories for efficient llm reasoning. *Advances in Neural Information Processing Systems*, 38:139724–139755.

Liang Wang, Nan Yang, Shaohan Huang, Li Dong, and Furu Wei. 2025. Thinking augmented pre-training. *arXiv preprint arXiv:2509.20186*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and 1 others. 2019. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024a. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2024b. Parallelizing linear transformers with the delta rule over sequence length. *Advances in neural information processing systems*, 37:115491–115522.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822.

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, and 1 others. 2026. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*.

A Additional Result Figures

Figures 5–10 show accuracy and parse rate results for the three task families not shown in the main body: State-based Astro Recall, Dyck Language, and OLMo Original.

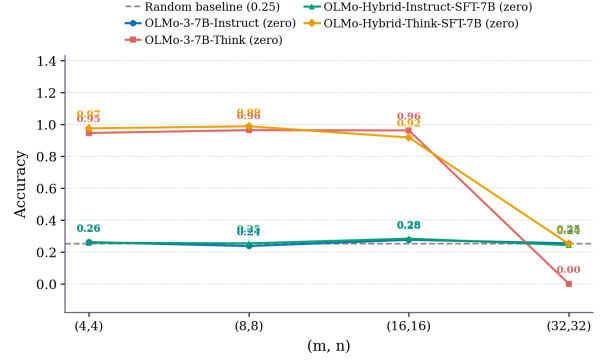


Figure 5: Astro Recall accuracy.

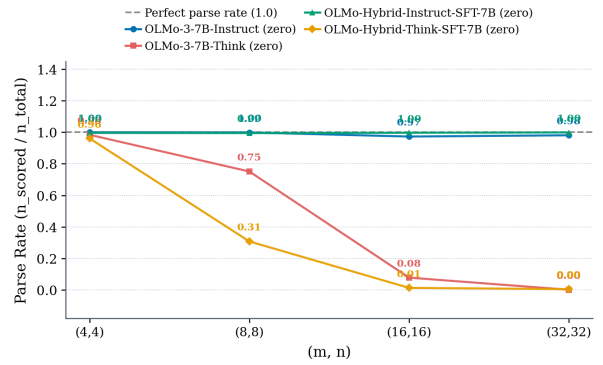


Figure 6: Astro Recall parse rate.

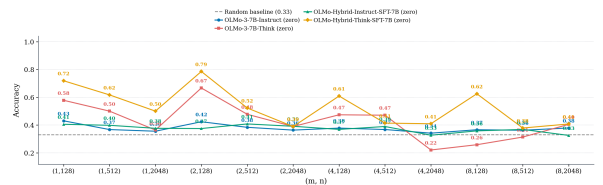


Figure 7: Dyck accuracy.

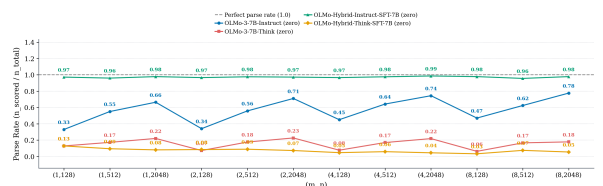


Figure 8: Dyck parse rate.

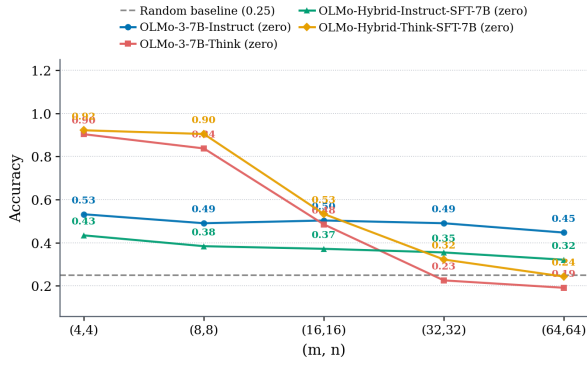


Figure 9: OLMo Original accuracy.

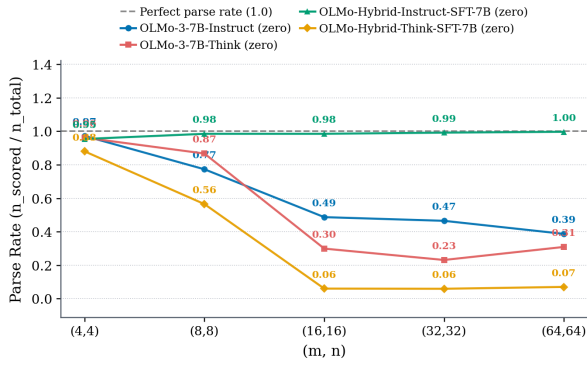


Figure 10: OLMo Original parse rate.

B Code Examples for Reasoning Primitives

The following examples illustrate the three task types discussed in Section 3: state-tracking, recall, and their composition into state-based recall.

Example 1. State-Tracking A canonical state-tracking task requires propagating variable bindings across a chain of dependent updates. The variables are initialized with binary values, then updated through a sequence of n dependent swap operations; the final assertion depends on the entire chain of updates. The challenge lies not in retrieving a fixed fact, but in preserving the result of many compositional updates—precisely the regime where recurrent architectures that maintain an incrementally updated hidden state are well matched:

```
a, b, c, d, e = [0, 1, 0, 0, 1]
a, c = c, e
...
assert a == _
```

Example 2. Recall A canonical recall task requires retrieving a specific item from a large array-like context given a precise index. The variable a holds an index into a long list of bits; the assertion depends solely on retrieving the bit at that posi-

tion. The challenge is to localize the relevant item without losing fidelity as context length grows—the regime where softmax attention functions as a differentiable associative memory mechanism, and where purely recurrent models that compress history into a fixed-dimensional state face increasing difficulty:

```
bits = [0, 1, 0, 0, ...]
a = 42
assert bits[a] == _
```

Example 3. State-Based Recall A canonical state-based recall task combines both primitives. A long list of bits serves as the memory, and the variables a – e are initialized as indices into that list. After n dependent swap operations, the final assertion depends on both correctly tracking the transformed index (state-tracking) and retrieving the bit at that position (recall). The retrieval step is conditioned on the outcome of the state-tracking step, making the task irreducible to either primitive in isolation. This is the regime where hybrid architectures are hypothesized to matter most: the recurrent components handle the sequential pointer transformations, while the attention mechanism supports the final precise lookup:

```
bits = [0, 1, 0, 0, ...]
a, b, c, d, e = 36, 23, 12, 2, 56
a, c = c, e
...
assert bits[a] == _
```

C Task Descriptions

This section provides full descriptions of all five task families summarised in Table 1 of the main text, ordered from simplest to most complex.

OLMo Original (*olmo_original*). A synthetic state-based recall task imported directly from (Merrill et al., 2026). A bit array of length m is generated and m pointer variables are each initialized to a distinct random index. After n simultaneous swap operations, the model must evaluate $\text{bits}[a]$ —combining pointer tracking with binary value lookup. This is the most minimal formulation of state-based recall in the suite and serves as the replication baseline.

State-based Astro Recall (*astro*). Each instance presents the model with a Markdown table of m real exoplanet rows. Named variables are initialised to the orbital period values of those rows, then subjected to n simultaneous swap operations. The model must determine which planet name corresponds to the queried variable after all swaps. This couples state-tracking (pointer propagation

through swaps) with recall (lookup in a semantically structured multi-attribute table), and tests whether the state-based recall challenge persists when embedded in a more naturalistic retrieval context.

Collision Simulator (*collisions*). Each instance places m particles in a one-dimensional system with distinct initial velocities drawn from a large integer pool. A sequence of n pairwise elastic collisions is applied in order; equal-mass particles exchange velocities on contact. Unlike the simultaneous swaps in the astro task, each collision here conditions all subsequent ones, creating a chain of compounding sequential dependencies. This makes the task more adversarial to state maintenance: a single error at step k corrupts all later steps.

Dyck Language (*dyck*). The model is given a bracket sequence drawn from the pairs $()$, $[\]$, $\{\}$ with one closing token masked, and must identify the correct closer by maintaining a stack. Difficulty is controlled by m (stack depth at the query position) and n (total sequence length). This is the only task in the suite containing no retrieval component—it stresses pure state-tracking in isolation.

DAG Arithmetic (*dag_arithmetic*). Following (Motwani et al., 2026), m input variables are assigned integer values and n computation layers introduce new variables via addition or subtraction, with later layers able to reference any prior variable. The flat, multi-hop retrieval structure over a growing variable registry places different demands on the model than the sequentially chained updates of the collision task—favouring content-addressable attention over recurrent state compression.

D Task Prompt Examples

For each task we show the zero-shot system prompt followed by a representative example prompt.

D.1 State-based Astro Recall

System Prompt

You are a precise reasoning assistant.

You will be given:

1. A table of exoplanet data with planet names and various properties.
2. Variable assignments mapping variable names to values from a specific column.
3. One or more swap operations. (Python-style simultaneous assignment)
4. A question asking which planet corresponds to a variable

after all swaps.

How to solve:

- Each variable (a , b , c , ...) starts assigned to a specific value from the table and therefore corresponds to a specific planet.
- Each swap line exchanges the values of two variables simultaneously:
 $x, y = y, x$ means x gets y 's current value and y gets x 's current value.
- Track which value each variable holds after every swap.
- At the end, find which planet in the table has the value that the queried variable currently holds.
- Apply every swap in order. Do NOT skip any.

Output requirements:

- Return EXACTLY one JSON object.
- No extra text.

Format:

```
{"answer": "A | B | C | D"}
```

Example Prompt

Planet	Orbital Period (days)
Kepler-1	2.684
Kepler-2	8.798
Kepler-3	5.508
Kepler-4	2.644

Consider the following Orbital Period (days):

$a, b, c, d = 2.684, 8.798, 5.508, 2.644$

Consider the following swapping:

- $a, b = b, a$
- $b, c = c, b$

The Planet with the Orbital Period (days) = a is:

Options

- A) Kepler-2
- B) Kepler-1
- C) Kepler-3
- D) Kepler-4

D.2 Collision Simulator

System Prompt

You are a strict state-tracking engine for collision systems.

Task:

- You are given particles with initial velocities.
- You are given a sequence of pairwise collisions.

Core rule (MUST be applied exactly):

- When two equal-mass particles collide, they EXCHANGE velocities.
- This is equivalent to swapping their velocity values simultaneously.
- If A collides with B: new_A = old_B, new_B = old_A. Both update at once.

Reasoning requirements:

- Maintain an explicit mapping: particle -> current velocity.
- Apply each collision in order.
- After each collision, update BOTH particles' velocities before moving to the next.
- Do NOT skip steps.
- Do NOT infer physics beyond the given rule.

Output requirements:

- Return EXACTLY one JSON object.
- No extra text.

Format:

```
{"answer": "A | B | C | D"}
```

Example Prompt

Physics Collision Task

Consider a one-dimensional system where all particles move along a line.

Key rule:

- When two equal-mass particles collide elastically, they exchange velocities.

Initial velocities

- A = 142
- B = 57
- C = 831

Collisions

1. A collides with B

2. B collides with C

Question

What is the velocity of particle A after all collisions?

Options

- A) 142
- B) 57
- C) 831
- D) 204

D.3 OLMo Original

System Prompt

You are a strict code-execution engine.

Task:

- You are given a bit array and pointer variables (a, b, c, ...).
- You are given a sequence of simultaneous swap assignments.
- You must track the pointer values through every swap and then look up the correct bit in the array.

Rules:

- Each swap line uses Python simultaneous assignment: x, y = y, x
- Apply every swap in order do not skip any.
- After all swaps, evaluate bits[<queried variable>].

Output requirements:

- Return EXACTLY one JSON object, no other text.

Format:

```
{"answer": "A | B | C | D"}
```

Example Prompt

```
bits = [0, 1, 1, 0] # 4 bits
a, b, c, d = 2, 0, 3, 1 # 0 to 3
a, b = b, a
b, c = c, b
assert bits[a] == _ # 0 or 1
```

Options

- A) 1
- B) 0
- C) 2
- D) 3

System Prompt

You are a strict language validator for Dyck expressions.

Rules:

- A Dyck expression uses bracket pairs: (), [], { }
- Every opening bracket must be closed by its exact matching closer.
- Brackets must be closed in the correct order (last opened = first closed).

Task:

- You are given a Dyck expression with one token masked as _.
- You must determine what token _ must be to keep the expression valid.

How to solve:

- Read the sequence token by token from left to right.
- Maintain a stack: push every opener ([{ onto the stack.
- When you see a closer)] }, pop the top of the stack.
- The masked token _ must match the opener currently on top of the stack at that position.
- Do NOT skip any token. Do NOT guess based on surrounding tokens alone.

Output requirements:

- Return EXACTLY one JSON object, no other text.

Format:

```
{"answer": "A | B | C"}
\end{codebox}
\begin{codebox}[Example Prompt]
\begin{verbatim}
Expression : ( [ { _ ] ) ( )

Bracket pairs: ( ) [ ] { }
Every opener must be closed by its matching closer in the correct order.

What token must replace _ at position 4?

### Options
A) )
B) }
C) ]
```

System Prompt

You are a strict arithmetic computation engine.

Task:

- You are given a set of input variables with integer values.
- You are given a sequence of computation steps organized in layers.
- Each step computes a new variable from one or two previous variables using addition or subtraction only.
- You must trace every computation in order and track all variable values.

Rules:

- Apply every step in order. Do not skip any.
- Use integer arithmetic throughout. All values are integers.
- Each step may reference any previously computed variable not just variables from the immediately preceding layer.
- Keep a running record of ALL variable values at all times.
- After all steps, report the value of the queried variable.

Output requirements:

- Return EXACTLY one JSON object, no other text.

Format:

```
{"answer": "A | B | C | D"}
```

Example Prompt

DAG arithmetic computation

Input variables:

```
a = 3
b = 7
```

Computation steps:

Layer 1:

```
Step 1: v1_0 = a + b
Step 2: v1_1 = b - 2
```

Layer 2:

```
Step 3: v2_0 = v1_0 + v1_1
Step 4: v2_1 = a + v1_0
```

What is the value of v2_1 after all

computations?

Options

- A) 10
- B) 13
- C) 15
- D) 7

E Formal Definitions of Reasoning Primitives

This appendix provides formal definitions of the two reasoning primitives introduced in Section 3 and motivates why their composition into state-based recall creates a computational challenge that neither primitive can address in isolation.

State-tracking. State-tracking is the ability to maintain and update structured variables over time under sequential transformations. The critical requirement is not merely the storage of initial values, but the preservation of the evolving relational configuration among variables as successive updates are applied. Formally, letting S_t denote the state at step t and U_t the update function applied at that step, the model must compute:

$$S_t = U_t(S_{t-1}) \quad (1)$$

and ensure that S_t accurately reflects the cumulative effect of all updates up to time t . Recurrent and state-space models are well matched to this regime because they maintain a compact hidden state updated incrementally at each step. Standard attention mechanisms are often less naturally aligned with problems whose difficulty lies in preserving the result of many compositional updates, especially when those updates are long-range, indirect, or involve repeated reassignment.

Recall. Recall is the ability to retrieve a specific piece of information from a large context with high precision, often when the relevant signal is sparse relative to the total input. Formally, given a context C of N items and a query q , the model must compute:

$$R = \text{Attention}(q, C) \quad (2)$$

such that R accurately reflects the relevant item in C corresponding to q , especially as N grows large and the relevant signal becomes sparser. This is the regime in which softmax attention is most naturally interpreted as a differentiable associative memory mechanism. Purely recurrent or linear state-space models must compress history into a fixed-dimensional state, making fine-grained retrieval progressively harder over long horizons.

State-based recall. When the two primitives are composed, the retrieval query is not a fixed

external input but is itself the output of a state-tracking process. The model must first compute the final state S_T via Equation 1 and then use that state as the query for retrieval via Equation 2:

$$R = \text{Attention}(S_T, C) \quad (3)$$

This composition makes the task irreducible to either primitive in isolation: an error in state propagation corrupts the retrieval address, and a failure in retrieval produces the wrong answer even given a correct address. Hybrid architectures are hypothesized to matter most in this regime precisely because they combine the inductive biases of both: recurrent components handle the sequential pointer transformations of Equation 1, while the attention mechanism supports the precise lookup of Equation 2.

F Infrastructure: Details

This appendix documents our compute infrastructure, software stack, and inference configuration.

F.1 HPC Cluster

All inference runs were carried out on the Marvin cluster, a tier 3 HPC facility hosted at the University of Bonn. Jobs were submitted to the ML-GPU partition, using a single node equipped with $8 \times$ NVIDIA A40 GPUs (48 GB HBM2 each), with intra-node GPU communication via NVLink and inter-node communication via Mellanox InfiniBand NDR at 200 Gb/s. Persistent storage was provided by a Lustre parallel filesystem with 5.6 PB total capacity.

F.2 Software Stack

Our codebase integrates the following libraries:

- **PyTorch** (Paszke et al., 2019) for distributed training via DDP and FSDP2.
- **Hugging Face Transformers** (Wolf et al., 2019) for model architectures.
- **FlashAttention-2** (Dao, 2023) for memory-efficient attention.
- **vLLM** (Kwon et al., 2023) for high-throughput inference and LLM-based evaluation.

F.3 Inference Configuration

All models were served using vLLM (Kwon et al., 2023) with a batch size of 8 and a maximum model context length of 28,000 tokens. Generation hyperparameters varied by model variant as shown in Table 2.

Parameter	Think variants	Instruct variants
Temperature	default	default
Max model len	28000	28000
Max tokens	4000	256
Batch size	8	8

Table 2: Inference generation parameters for Think and Instruct model variants.

G Model Architecture Details

Table 3 summarises the architectural properties shared across all four models evaluated in this study, and Table 4 highlights the key differences between the transformer and hybrid model families.

Property	Value
Parameters	7B
Training data	Dolma 3
Post-training data	Dolci
Training tokens	6T
License	Apache 2.0
Inference framework	vLLM
Post-training stages	SFT → DPO → RLVR

Table 3: Architectural and training properties shared across all four models.

Property	OLMo3 former)	(Trans- former)	OLMo3 Hybrid
Architecture type	Full transformer		Hybrid (attention + recurrent)
Recurrent layer	None		Gated DeltaNet (GDN)
Layer pattern	32× attention		3× GDN + 1× attention, repeated
Attention fraction	100%		25%
Hidden size (d_{model})	4096		3840
Attention heads	32		30
Head dimension	128		256 (doubled for DeltaNet)
Positional encoding	RoPE		RoPE disabled
Context length	Standard		Extended long-context
Variants evaluated	Instruct, Think		Instruct-SFT, Think-SFT

Table 4: Key architectural differences between the OLMo3 transformer and OLMo3 Hybrid model families. Both families are matched on training data, optimization schedule, and post-training recipe.